

What we are building is essentially an **AI-assisted SDLC operating system for SaaS product development**: one web interface that takes a product from **Idea** → **Requirements** → **Architecture** → **Design** → **Code** → **Test** → **Release** → **Production**, while preserving complete traceability.

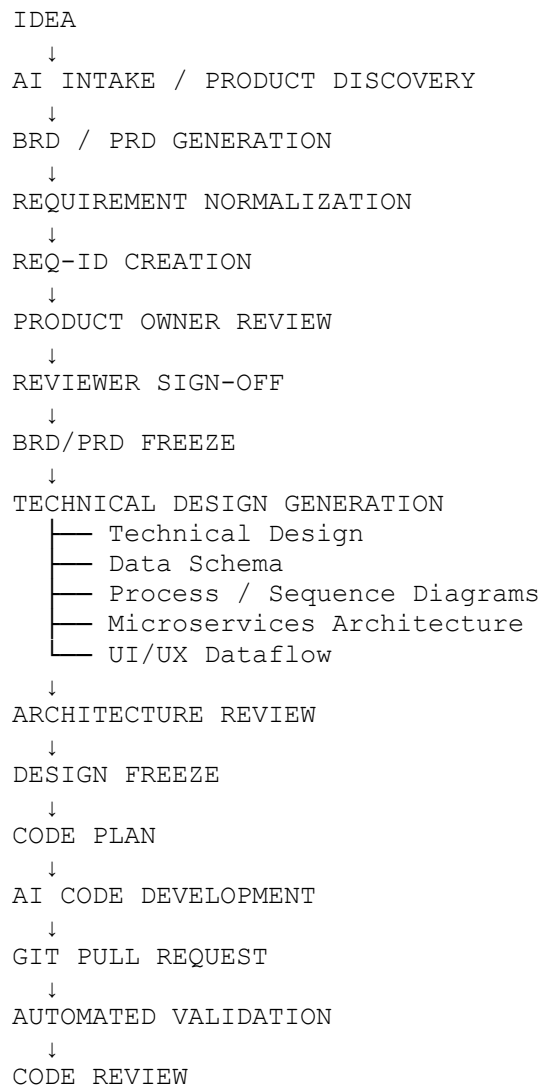
The most important architectural principle I would recommend is:

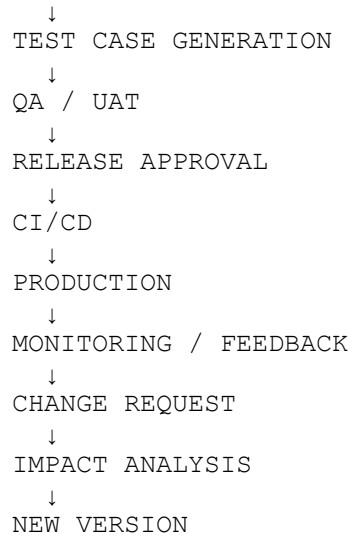
REQ-ID should become the immutable spine of the entire product lifecycle.

Everything else—documents, APIs, database entities, diagrams, UI screens, code, test cases, defects and releases—should be traceable back to one or more REQ-IDs.

1. Recommended overall process

I would formalize your process into the following lifecycle:





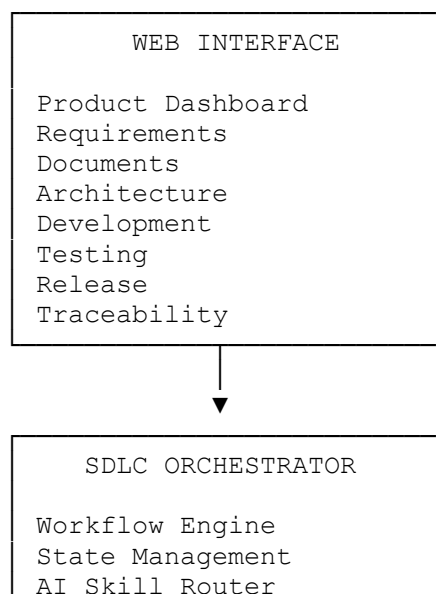
The important addition to your current process is **Requirement Normalization + Traceability + Change Impact Analysis**.

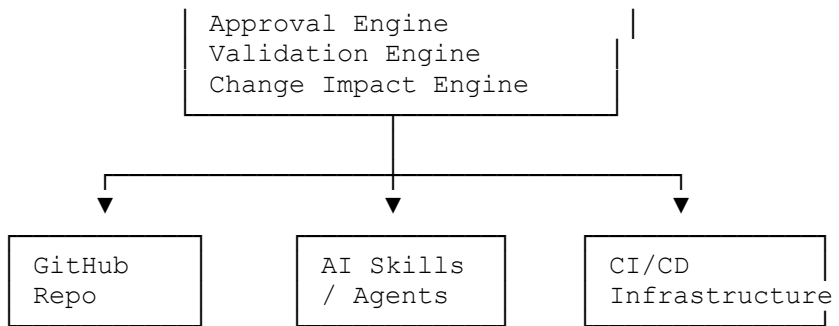
Those three capabilities will eventually become the most valuable part of your platform.

2. Think of the system as three layers

Your web interface should not directly "do everything."

Use this architecture:





This separation is extremely important.

The **Web UI** is the control plane.

The **AI Skills** are workers.

GitHub is the **code/document source repository**.

The workflow engine is the **brain controlling progression between stages**.

3. Your REQ-ID needs to become much more powerful

Currently you are thinking:

```
REQ-001
REQ-002
REQ-003
```

I would turn that into a formal requirement object.

For example:

```
REQ-ID: AUTH-001
```

```
Product: SaaS-PRODUCT-001
```

```
Module: Authentication
```

```
Requirement:
```

```
    User shall be able to login using email and password.
```

```
Type:
```

```
    Functional
```

Priority:
High

Acceptance-Criteria:
- Valid email/password allows login
- Invalid password displays error
- Account locks after configurable attempts

Dependencies:
- AUTH-002
- USER-001

Status:
APPROVED

Version:
1.0

Owner:
Product Owner

Approved-By:
Reviewer

Approved-Date:
2026-09-23

But the really important part comes next.

The same AUTH-001 should appear across the entire SDLC.

For example:

AUTH-001

- BRD Section
- PRD Section
- TDD-AUTH-001
- API-AUTH-001
- DB-ENTITY-USER
- SERVICE-AUTH
- UI-SCREEN-LOGIN
- TEST-AUTH-001-01
- TEST-AUTH-001-02
- CODE PR

Then your platform can answer:

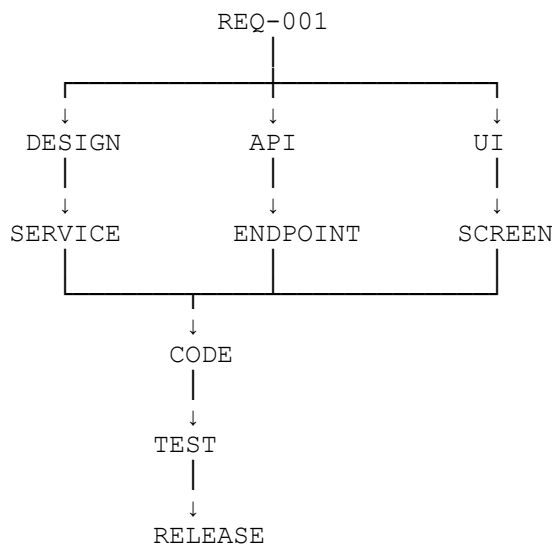
"Show me everything affected if AUTH-001 changes."

That is enormously powerful.

4. Introduce a Requirement Traceability Graph

I would actually implement a **graph**, not merely a flat traceability table.

Conceptually:



This gives you a **digital thread through the product**.

Your interface can then show:

REQ-001

BRD	✓ Approved
PRD	✓ Approved
Technical Design	✓ Approved
API	✓ Defined
Database	✓ Defined
UI	✓ Defined

Code	✓ Implemented
Tests	5/5 Passed
UAT	✓ Passed
Production	✓ Deployed

That becomes the central dashboard for your platform.

5. Your BRD/PRD stage should be slightly different

Your current flow is:

```
Idea
↓
ChatGPT
↓
Custom Skill
↓
BRD/PRD
```

I would make it:

```
RAW IDEA
↓
AI PRODUCT DISCOVERY
↓
CLARIFICATION ENGINE
↓
REQUIREMENT EXTRACTION
↓
REQUIREMENT NORMALIZATION
↓
BRD / PRD
↓
REVIEW
↓
APPROVAL
↓
FREEZE
```

The **Clarification Engine** is important.

For example, if the user says:

"Build an employee attendance SaaS."

The AI should not immediately create the final PRD.

It should identify unresolved items:

Questions requiring business decision:

1. Web only or mobile?
2. Biometric attendance required?
3. Multi-tenant?
4. Role hierarchy?
5. Payroll integration?
6. Geolocation?
7. Shift management?
8. Attendance correction workflow?
9. Leave management?
10. Expected number of employees?

The user answers these inside your UI.

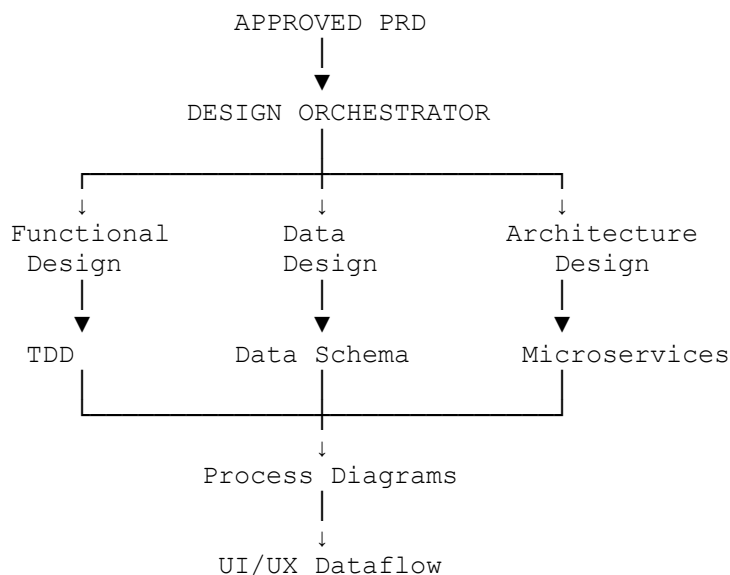
Then the AI generates the PRD.

That substantially improves the downstream technical design.

6. After BRD/PRD freeze, use a Design Generator Pipeline

Your five-document concept is very good.

I would formalize it as:



Your standard design pack becomes:

01 — Technical Design Document

- REQ → Functional behaviour
- REQ → Business logic
- REQ → API contract
- REQ → Error handling
- REQ → Validation
- REQ → Security requirements

02 — Data Schema

```

REQ
↓
Entity
↓
Attributes
↓
Relationships
↓
Indexes
↓
Constraints

```

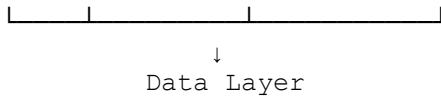
03 — Process / Object Flow

For example:

```
graph TD
    User --> LoginUI[Login UI]
    LoginUI --> APIGateway[API Gateway]
    APIGateway --> AuthService[Auth Service]
    AuthService --> UserDB[(User DB)]
    UserDB --> TokenService[Token Service]
    TokenService --> Response
```

04 — Microservices Architecture

```
graph TD
    Frontend --> APIGateway[API Gateway]
    APIGateway --> Auth
    APIGateway --> User
    APIGateway --> Billing
    APIGateway --> Notification
```



05 — UI/UX Dataflow

This should go beyond screen mockups.

For every screen:

```
SCREEN
↓
USER ACTION
↓
UI STATE
↓
API CALL
↓
BACKEND SERVICE
↓
DATABASE
↓
RESPONSE
↓
UI STATE CHANGE
```

That makes it much easier for the coding engine to generate consistent code.

7. Add one more document: TEST DESIGN

I would strongly recommend adding a sixth standard artifact.

06 — Test Design

Each requirement should generate tests.

For example:

REQ-AUTH-001

Test:

AUTH-001-T01

Valid username/password

AUTH-001-T02

Invalid password

AUTH-001-T03

Locked account

AUTH-001-T04
Session expiry

AUTH-001-T05
Brute force protection

Then you get:

```
REQ
↓
DESIGN
↓
CODE
↓
TEST
```

This lets your system determine:

"Requirement implemented but not tested."

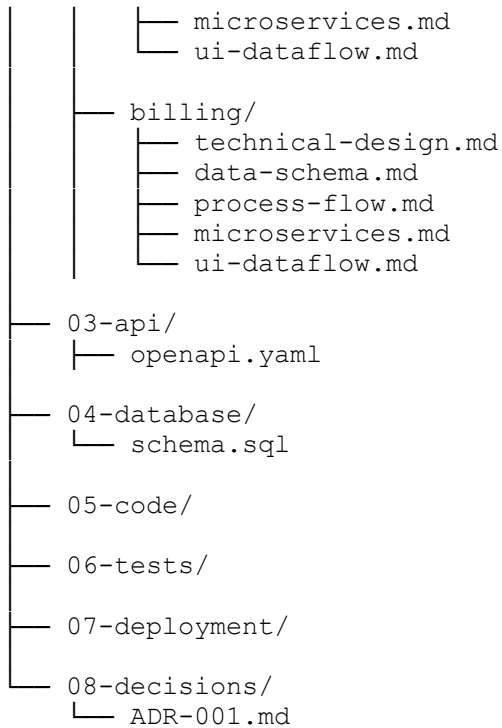
That should be visible directly on the dashboard.

8. The GitHub repository structure

I would recommend a consistent docs-as-code repository.

For example:

```
product-name/
├── 00-product/
│   ├── vision.md
│   ├── product-charter.md
│   └── roadmap.md
├── 01-requirements/
│   ├── brd.md
│   ├── prd.md
│   ├── requirements.yaml
│   └── traceability.yaml
├── 02-design/
│   └── authentication/
│       ├── technical-design.md
│       ├── data-schema.md
│       └── process-flow.md
```



And every file contains the relevant requirement IDs.

For example:

```
# Authentication Technical Design
```

```
Requirements:
```

- AUTH-001
- AUTH-002
- AUTH-003

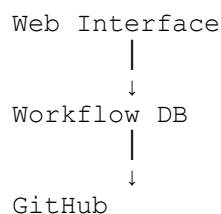
```
Status: APPROVED
```

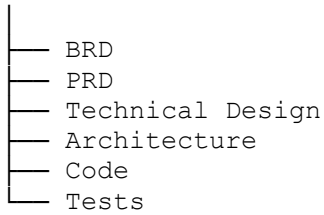
```
Version: 1.2
```

9. GitHub should remain the source of truth

Your web application should **not** become another document repository.

I recommend:





The web interface should display and orchestrate the GitHub content.

This gives you:

- version control
- commits
- branches
- pull requests
- approvals
- audit trail
- rollback
- diffs
- history

without reinventing Git.

10. Your web interface should look like a product cockpit

I would not create 20 disconnected screens.

Make the platform revolve around a **Product Workspace**.

For example:

PRODUCT: Employee Attendance SaaS			Version 1.0	
IDEA	REQUIREMENTS	DESIGN	CODE	RELEASE
✓	✓	85%	40%	-
MODULES				
Authentication			100%	
Employee Management			80%	
Attendance			50%	

Payroll		20%
REQUIREMENT HEALTH		
Total Requirements	138	
Approved	130	
Design Complete	115	
Code Complete	87	
Tested	72	
UAT Passed	65	

11. I would create these main sections

Product Dashboard

Overall health.

Requirements

REQ-ID
Description
Module
Priority
Status
Owner
Dependencies

Clicking a requirement should open its entire lifecycle.

Document Studio

Shows:

BRD
PRD
Technical Design
Schema
Architecture
Process Flow
UI Dataflow
Test Design

with version history and approval status.

Architecture Studio

This should show:

Services
APIs
Databases
Events
Queues
External Systems
Dependencies

and eventually provide interactive diagrams.

Development

Show:

Requirements → Tasks → Git Branch → PR → Code → Build

Testing

Show:

Requirements
↓
Test Cases
↓
Automation
↓
Execution
↓
Defects

Release

DEV
↓
QA
↓
UAT
↓
STAGING
↓
PRODUCTION

Traceability

This could become one of your platform's strongest screens.

12. The "Requirement 360" screen

Imagine clicking:

AUTH-001

The screen should show:

AUTH-001
User login

STATUS

Requirement	✓
BRD	✓
PRD	✓
Technical Design	✓
API	✓
Database	✓
UI	✓
Code	✓
Tests	✓
UAT	✓
Production	✓

DEPENDENCIES
AUTH-002
USER-001

CODE
PR #284
Commit: 93ab2c

TESTS
5 / 5 Passed

RELEASE
Release 1.4.0

This is the point where your platform stops being merely an **AI document generator** and becomes a true **SDLC management platform**.

13. Your "Skills" need standard contracts

I would avoid having arbitrary prompts like:

Skill 1
Skill 2
Skill 3

Instead define every skill using a standard contract.

For example:

BRD Generator

Input:
Raw Idea
User answers
Existing product context

Preconditions:
Idea exists

Output:
BRD
PRD
Requirements

Validation:
Requirement completeness
Duplicate requirements
Ambiguity detection

Artifacts:
BRD
PRD
requirements.yaml

Then:

Technical Design Generator

Input:
Approved PRD
Requirements

Output:
Technical Design
Data Schema
Process Flow
Microservice Design
UI Dataflow

Precondition:
PRD = APPROVED

Then:

Code Generator

Input:
Approved Design
Approved API
Approved Schema

Output:
Code
Unit Tests
Documentation

Precondition:
Design = APPROVED

That lets you scale to dozens of AI skills without making the platform unmanageable.

14. Use a state machine

This is one of the most important parts of your implementation.

Do not let the UI simply have buttons like:

Generate next document.

The system should understand states.

For example:

```
IDEA
↓
DRAFT
↓
REVIEW
↓
APPROVED
↓
FROZEN
```

For design:

```
DESIGN-DRAFT
↓
DESIGN-REVIEW
↓
DESIGN-APPROVED
↓
DESIGN-FROZEN
```

For development:

```
PLANNED
↓
IN-DEVELOPMENT
↓
CODE-REVIEW
↓
MERGED
↓
TESTING
↓
UAT
↓
RELEASED
```

The next AI operation should depend on state.

For example:

```
IF PRD != APPROVED
  → Do not generate production code
```

and:

```
IF DESIGN != APPROVED
  → Code generation unavailable
```

That creates governance around AI.

15. The approval mechanism

I recommend **GitHub PR + platform approval** together.

Example:

```
AI generates PRD
↓
Creates Git branch
↓
Pull Request
↓
Reviewer comments
↓
AI revision
↓
Reviewer approves
↓
Product Owner approves
```

Merge
↓
SYSTEM STATE = PRD_FROZEN

The platform should display:

Approval Matrix

Product Owner	✓
Technical Reviewer	✓
Security Reviewer	-
Architecture	-

depending on the stage.

16. Very important: change management

This is where the system can become exceptionally powerful.

Suppose:

REQ-015
"Customer can pay by credit card."

has already reached production.

Now Product Owner changes it to:

"Customer can pay by credit card and UPI."

The system should automatically identify:

REQ-015
↓
BRD impacted
↓
PRD impacted
↓
Payment Design impacted
↓
Payment Service impacted
↓
Database possibly impacted
↓
API impacted
↓
UI payment page impacted

↓
Tests impacted
↓
Release impacted

Then show:

CHANGE IMPACT

12 artifacts affected
3 services affected
2 APIs affected
1 database entity affected
8 tests affected
1 production release affected

This should become a **core feature**, not an afterthought.

17. Recommended backend data model

At minimum I would create these entities:

Product
Project
Module
Requirement
RequirementVersion
Artifact
ArtifactVersion
Approval
User
Role
Skill
Workflow
WorkflowExecution
Dependency
Service
API
DatabaseEntity
Screen
TestCase
Defect
Release
Deployment
ChangeRequest
AuditEvent

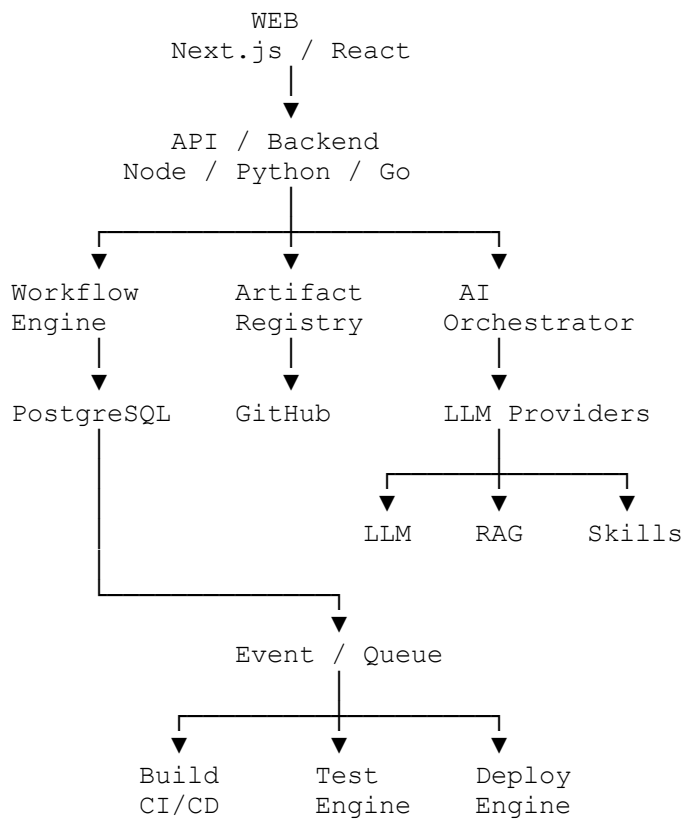
The important relationship is:

Requirement

- Artifact
- API
- Service
- DB Entity
- Screen
- Test
- Code PR
- Release

18. Suggested technical architecture for your platform

A practical architecture could be:



For an enterprise-grade version, a workflow engine such as **Temporal** can be very useful because your SDLC processes are long-running, stateful workflows with retries, human approvals and asynchronous jobs.

19. AI should operate through "Context Packs"

One problem you will eventually face is:

"How do I give an AI skill enough context?"

Do not simply send the entire repository to every model call.

Create a **Context Assembly Engine**.

For example, when generating the authentication Technical Design:

Context Pack

PRD sections:

REQ-AUTH-001

REQ-AUTH-002

REQ-AUTH-003

Relevant business rules

Existing architecture:

API Gateway

User Service

Existing APIs:

POST /login

POST /logout

Existing entities:

User

Session

Existing design decisions:

ADR-003

Security standards:

JWT

RBAC

Then the skill receives only the relevant context.

That will make your AI processing:

- cheaper
- faster
- more deterministic

- less prone to hallucination
- easier to audit

20. Add an AI Validation layer

This is another major recommendation.

Never rely on:

AI generates → accept

Instead:

```
AI generates
    ↓
AI Validator
    ↓
Rule Validator
    ↓
Consistency Validator
    ↓
Human Review
```

For example, your validator could detect:

REQ says:
"System must support 100,000 users."

Architecture says:
single-instance database.

⚠ Potential architecture inconsistency

Or:

REQ-023 requires OTP.

UI design has no OTP screen.

⚠ Traceability gap

Or:

API /payment/refund

Technical Design:
POST

OpenAPI:
GET

⚠ Contract inconsistency

This is where AI becomes much more useful than simply generating text.

21. Introduce a "Quality Gate"

Every stage should have measurable gates.

For example:

PRD Gate

Requirements complete	✓
Acceptance criteria	✓
Duplicate requirements	✓
Ambiguity	2 warnings
Dependencies	✓
Approval	✓

Technical Design Gate

REQ coverage	98%
API defined	✓
Schema defined	✓
Architecture defined	✓
Security review	✓
Data consistency	✓

Code Gate

REQ coverage	96%
Unit tests	87%
Static analysis	✓
Dependency scan	✓
Security scan	✓
Build	✓

Release Gate

Critical defects	0
High defects	0

UAT	✓
Security	✓
Performance	✓
Rollback	✓
Production approval	✓

The platform can then show:

READY FOR NEXT STAGE

rather than simply:

NEXT

22. I would design the workflow around "Gates", not documents

This is subtle but important.

Your current thinking is:

```
Document 1
↓
Document 2
↓
Document 3
↓
Document 4
```

The stronger design is:

```
BUSINESS GATE
↓
DESIGN GATE
↓
DEVELOPMENT GATE
↓
QUALITY GATE
↓
RELEASE GATE
```

Documents are evidence supporting each gate.

That makes the platform much more robust.

23. The complete UI could therefore look like this

AI SaaS PRODUCT FACTORY					
Portfolio					
Product A	Product B	Product C			
78%	42%	91%			
PRODUCT A					
IDEA → REQUIREMENTS → DESIGN → CODE → TEST → RELEASE					
✓	✓	✓	62%	34%	-
Modules					
Authentication	100%				
User Management	82%				
Billing	68%				
Reporting	41%				
ACTIONS					
Review 7 Requirements					
Approve Technical Design					
Resolve 3 Architecture Conflicts					
Review Pull Request #284					
Execute UAT					

24. I would divide implementation into 3 phases

Phase 1 — MVP

Build only:

Idea

↓
BRD/PRD
↓
REQ-ID
↓
Approval
↓
Technical Design Pack
↓
GitHub

Plus:

- Product dashboard
- Requirement explorer
- Document viewer
- GitHub integration
- Approval workflow
- Traceability

This proves your core concept.

Phase 2 — Engineering Automation

Add:

Design
↓
API
↓
Database
↓
Architecture
↓
Code Plan
↓
Code Generation
↓
PR
↓
CI/CD

Then add:

- Test generation
- Automated validation
- AI code review
- Security checks
- Requirement-to-code traceability

Phase 3 — Autonomous SDLC Platform

Then move toward:

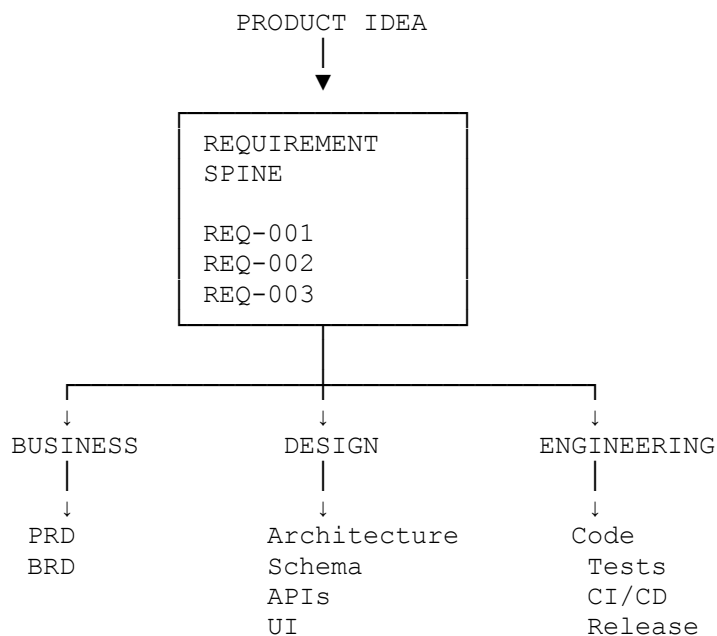
```
Idea
↓
AI Product Manager
↓
AI Architect
↓
AI Developer
↓
AI Tester
↓
AI Release Manager
```

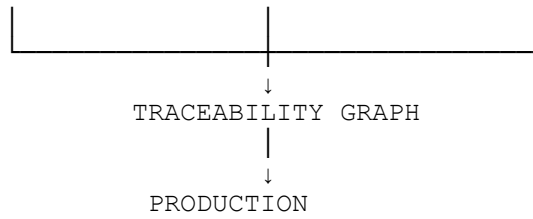
with humans acting primarily at **decision and approval gates**.

At that point, the platform becomes much more than a document-generation system.

25. The most important architectural principle

I would summarize your platform as:





The **REQ-ID graph + state machine + AI skill orchestration + GitHub integration** should therefore be the foundation of your system.

The documents themselves are outputs of that system.